



AULA 2

Sistemas NoSQL e BigData



Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- Identificar as abordagens NoSQL para gerenciamento e manuseio de dados.
- Descrever as categorias principais de sistemas NoSQL: chave-valor, orientado a documentos ou colunas e baseados em grafos.
- Distinguir um sistema NoSQL baseado em documentos.
- Explorar um sistema NoSQL baseado em grafos.
- Praticar linguagens e acessos de sistemas não relacionais.

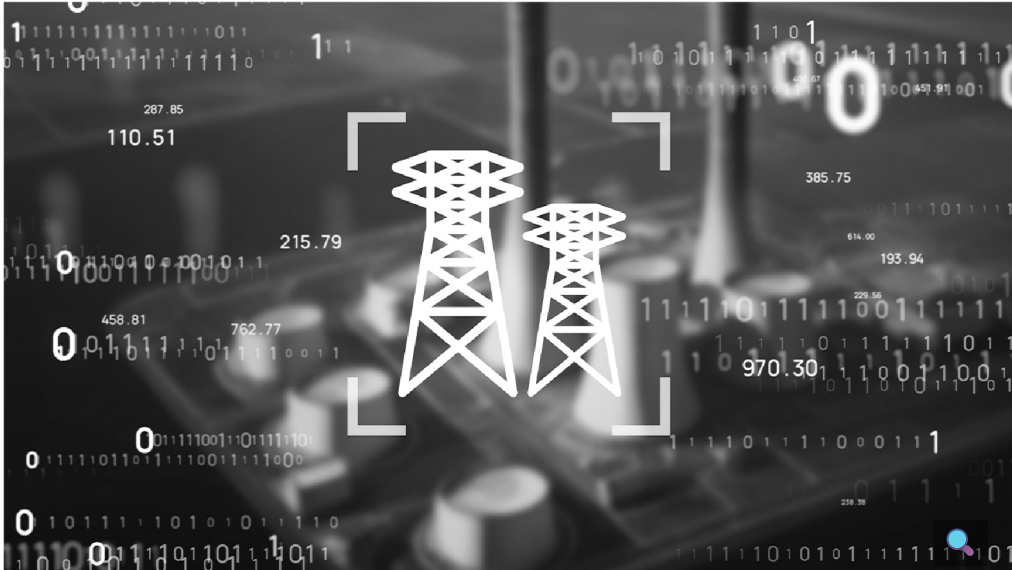
Que história é essa?

Na aula anterior, estudamos os *data warehouses*, seguindo, de certa forma, o modelo de dados relacional, hoje considerado convencional e com algumas limitações que podem não atender aos requisitos de novas aplicações — por exemplo, a ausência de esquemas predefinidos, a escalabilidade horizontal e, principalmente, a adequação aos novos tipos de representação de dados.

Neste módulo, você vai aprender um pouco mais sobre os chamados sistemas NoSQL, que, na prática, são sistemas que não seguem o modelo relacional que conhecemos, trazendo novas estruturas, representações e linguagens. Vamos também estudar e conhecer alguns sistemas mais populares, em particular, os modelos orientados a documentos e aqueles baseados em grafos.

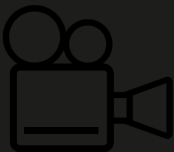
O termo NoSQL surgiu em 1998 e pode significar que um sistema gerenciador de banco de dados (SGBD) não utiliza a linguagem SQL ou não segue as regras definidas pelo modelo relacional de dados.

Um SGBD relacional implementa o modelo relacional de dados. Segundo esse modelo, os dados são organizados em tabelas, que, por sua vez, têm linhas e colunas. De modo a descrever as tabelas de um banco de dados relacional, existe um dicionário de dados, também conhecido como metadados ou esquema, que é definido antes do armazenamento dos dados.



Em um esquema, estão todas as restrições de integridade a que os dados devem obedecer. Outra característica dos SGBDs relacionais é que eles utilizam a linguagem SQL para criação de tabelas, consulta e atualização de dados. Os SGBDs relacionais também priorizam as propriedades ACID, de modo a garantir a consistência dos dados a qualquer instante.

Uma característica comum aos SGBDsNoSQL é que, em vez de priorizar a consistência, eles priorizam uma disponibilidade dos dados. Outro ponto muito importante é que, no SGBDNoSQL, não é necessário definir o esquema de dados *a priori*, sendo, dessa forma, conhecido como “sem esquema”, ou *schemaless*, em inglês.



Características dos sistemas NoSQL

Muitas das características dos sistemas NoSQL surgiram juntamente com as demandas dos chamados sistemas *big data*, por conta de exigências tecnológicas antes inexistentes. Cabe, neste momento, fazermos uma pausa para definirmos e esclarecermos esse novo contexto tecnológico. Para isso, recomendamos que você assista a esta breve videoaula.



Sistemas NoSQL

Os SGBDsNoSQL geralmente são diferenciados pelos tipos de modelos de dados a que dão suporte e são divididos em categorias. Clique para conhecê-las:

Interativo

Descrição do interativo

Existem também os chamados sistemas multimodelo (ou *polystore*), que suportam mais de um desses modelos de dados. Cabe observar que muitos SGBDsNoSQL não têm todas as características presentes em SGBDs relacionais — os mais conhecidos no mercado.

Assim, alguns autores e profissionais preferem chamá-los de NoSQLStores ou sistemas NoSQL, para diferenciá-los dos sistemas que implementam todas as funcionalidades presentes nos SGBDs relacionais. Como (ainda) não há um termo largamente adotado, por conta das novidades tecnológicas, vamos seguir utilizando SGBDNoSQL neste texto.

NoSQL chave-valor

Um dos modelos NoSQL mais simples é conhecido como chave-valor. Todo e qualquer dado armazenado nesse modelo recebe uma chave atribuída pelo SGBD no momento em que é inserido no banco de dados. Associado a essa chave, pode ser armazenado um valor, que pode ser simples ou estruturado.



Para um SGBD chave-valor, os valores são “uma caixa preta”, no sentido de que não há nenhuma validação ou interpretação sobre os valores armazenados — ou seja, não há semântica atribuída a esses valores que seja de conhecimento do SGBD. A semântica dos valores armazenados é única e exclusivamente entendida pelos programas que manipulam esses dados.

Não há uma linguagem para recuperação de dados, pois toda a recuperação ocorre com base na informação de uma chave de acesso ao SGBD. Dessa forma, os comandos são muito simples; por exemplo: SET (cria uma chave e seu respectivo valor), GET (recupera,

a partir da informação de uma chave, o valor associado) e DEL (remove do banco de dados uma chave e seu respectivo valor associado). Um exemplo de SGBD chave-valor é o Redis (Remote Dictionary Server), criado em 2009.

Modelos semiestruturados

Uma evolução do modelo NoSQL muito simples é aquela que permite associar a uma chave um valor que tem uma estrutura informada no momento da criação do par chave-valor. Nesse caso, afirmamos que há metadados, porém eles não são definidos antes do armazenamento dos dados.



Nesses modelos semiestruturados, é comum que o valor associado à chave seja informado na forma de um texto que tem não apenas os dados, mas também a sua descrição.

Existem duas linguagens muito conhecidas e consolidadas que são utilizadas para informar dados e sua descrição: XML e JSON. Essas linguagens surgiram não em um contexto de persistência de dados, mas, sim, de interoperabilidade, assim definida pelo ePING (Padrões de Interoperabilidade de Governo Eletrônico):

“A interoperabilidade pode ser entendida como uma característica que se refere à capacidade de diversos sistemas e organizações trabalharem em conjunto (interoperar) de modo a garantir que pessoas, organizações e sistemas computacionais interajam para trocar informações de maneira eficaz e eficiente.”

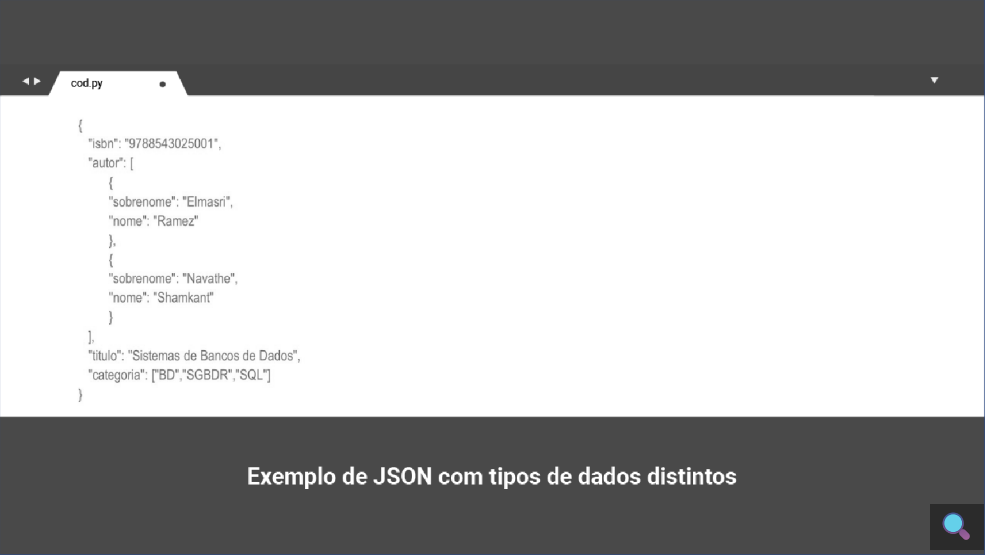
NoSQL orientado a documentos

Um modelo semiestruturado NoSQL muito conhecido é o chamado orientado a documento. Os SGBDs orientados a documento têm “coleções”, que são conjuntos de documentos. É comum que nesses SGBDs os valores sejam armazenados em formato JSON (Javascript Object Notation).

O exemplo da figura a seguir apresenta uma representação simples em JSON de características, por meio de pares “campo: valor”, de uma espécie de árvore:



Outro exemplo com dados atômicos, não atômicos e estruturados é apresentado na próxima figura. Há dois dados não atômicos (com mais de um valor) que são “autor” e “categoria”, sendo que cada elemento de autor é estruturado, pois tem “sobrenome” e “nome”.



Diferentemente de um SGBD chave-valor, para um SGBD orientado a documento, os valores não são “uma caixa preta”, pois a sua descrição é armazenada junto aos valores. Nesse caso, o SGBD tem conhecimento das informações de cada documento armazenado. Embora não haja uma linguagem para recuperação de dados padronizada, como a linguagem SQL para os SGBDs relacionais, é possível recuperar documentos com base em expressões lógicas que utilizam as características de um documento.

O armazenamento de documentos em formato JSON traz flexibilidade à persistência de dados, pois cada documento pode ter suas próprias características. Mas se, por um lado, isso permite flexibilidade ao esquema (que não precisa ser definido *a priori*), pode ser uma dor de cabeça, pois, ao recuperar um documento, um programa não sabe *a priori* quais são as suas características.



Por exemplo, em uma mesma coleção de livros, um livro A pode ter título e categoria, enquanto outro livro B pode ter apenas o nome de seus autores e o endereço *web* de onde ele pode ser comprado.

Sistema MongoDB

Um exemplo de SGBD orientado a documento é o MongoDB, criado em 2009 e que se tornou muito popular por sua robustez e suas

características tecnológicas. O MongoDB não tem uma linguagem, como SQL, e a manipulação de dados precisa ser realizada por meio de métodos que toda coleção apresenta. Há métodos para criação de documentos (insertOne, insertMany), para busca de dados (find), para atualização de dados (updateOne, updateMany, replaceOne) e para remoção de dados (deleteOne, deleteMany). Os parâmetros informados para esses métodos seguem o formato definido pela linguagem JSON.

No método insertOne, é necessário informar a coleção em que o documento deve ser inserido. Sua função é a mesma do comando INSERT da linguagem SQL. O documento em si deve ser informado segundo a sintaxe JSON.

Na figura a seguir, a coleção se chama “users”, e o documento que está sendo inserido tem os campos “name”, “age” e “status”.

cod.py

```
db.users.insertOne(  
  {  
    name: "sue",  
    age: 26,  
    status: "pending"  
  }  
)
```

← collection

← field: value

← field: value

← field: value

} document

Exemplo do método insertOne

Fonte: <https://www.mongodb.com/docs/manual/crud/>.

Observe a figura a seguir. No método find, devem ser citados a coleção (“users”) e o critério de busca (“age”, acima de 18). Caso a opção seja por apresentar apenas alguns campos dos documentos encontrados, estes devem ser informados (“name” e “address”, no exemplo), como em uma operação de projeção de álgebra relacional.

Por fim, alguns modificadores também podem ser informados. No exemplo da mesma figura, foi informado ao MongoDB para que ele retorne apenas os cinco primeiros documentos encontrados. A função do find é a mesma do comando SELECT da linguagem SQL.

cod.py

```
db.users.find(  
  { age: { $gt: 18 } },  
  { name: 1, address: 1 }  
).limit(5)
```

← collection

← query criteria

← projection

← cursor modifier

Exemplo do método find

Fonte: <https://www.mongodb.com/docs/manual/crud/>.

O método updateMany permite atualizar vários documentos de uma só vez. Sua função é a mesma do comando UPDATE da linguagem SQL. No exemplo apresentado na figura, todos os documentos da coleção “users” em que a idade for menor que 18 terão o seu campo “status” atualizado para “reject”.

cod.py

```
db.users.updateMany(  
  { age: { $lt: 18 } },  
  { name: 1, address: 1 }  
  { $set: { status: "reject" } }  
)
```

← collection

← update filter

← update action

Exemplo do método updateMany

Fonte: <https://www.mongodb.com/docs/manual/crud/>

Por fim, o método deleteMany permite remover vários documentos de uma só vez. Sua função é a mesma do comando DELETE da linguagem SQL. No exemplo apresentado na figura a seguir, todos os documentos da coleção “users” em que o “status” tiver o valor “reject” serão removidos da coleção.

cod.py

```
db.users.deleteMany(  
  { status: "reject" }  
)
```

← collection

← delete filter

Exemplo do método updateMany

Fonte: <https://www.mongodb.com/docs/manual/crud/>

NoSQL orientado a grafos

Você sabe o que são grafos? Descubra no *podcast* a seguir.



Grafos

A imagem a seguir representa o problema levantado por Euler apresentado no *podcast*. Você consegue imaginar uma solução possível?



Grafos e aplicações práticas: ilustração da cidade de Konigsberg

Para saber mais, realize a leitura do artigo “Leonard Euler’s solution to the Königsberg Bridge problem”, de Teo Paoletti (em inglês).

E como são armazenados esses grafos, em caso de grandes volumes de dados? É o que veremos a seguir.

SGBD orientado a grafos

O armazenamento de grandes volumes requer a utilização de SGBDs. Assim, os SGBDs que suportam modelos de dados em grafos ganharam destaque novamente no movimento NoSQL.

Neste módulo, vamos utilizar o sistema Neo4J, um dos sistemas mais bem colocados em *rankings* internacionais, que conta com instalação local ou na nuvem. Primeiramente, vamos apresentar os modelos de dados em grafo mais populares (LPG e RDF) e as linguagens de manuseio associadas a esses modelos.

Grafos em modelos de dados

Um modelo de dados especifica como as entidades do mundo real e seus relacionamentos são representadas e operadas. Porém, dentro da categoria de banco de dados em grafo, ainda podemos encontrar dois modelos mais usados: grafo de propriedades rotuladas (LPG) e estrutura de descrição de recursos (RDF). Esses modelos têm em comum elementos herdados da definição matemática de grafos. Confira nos *cards* a seguir:

Interativo

Descrição do interativo

Alguns SGBDs, como Neo4j, DEX, TigerGraph e InfiniteGraph, dão suporte ao modelo LPG, enquanto o RDF foi adotado por outros bancos de dados de grafos, como Oracle Database Spatial and Graph, OpenLink Virtuoso e AllegroGraph, também conhecidos como *triplestores*.

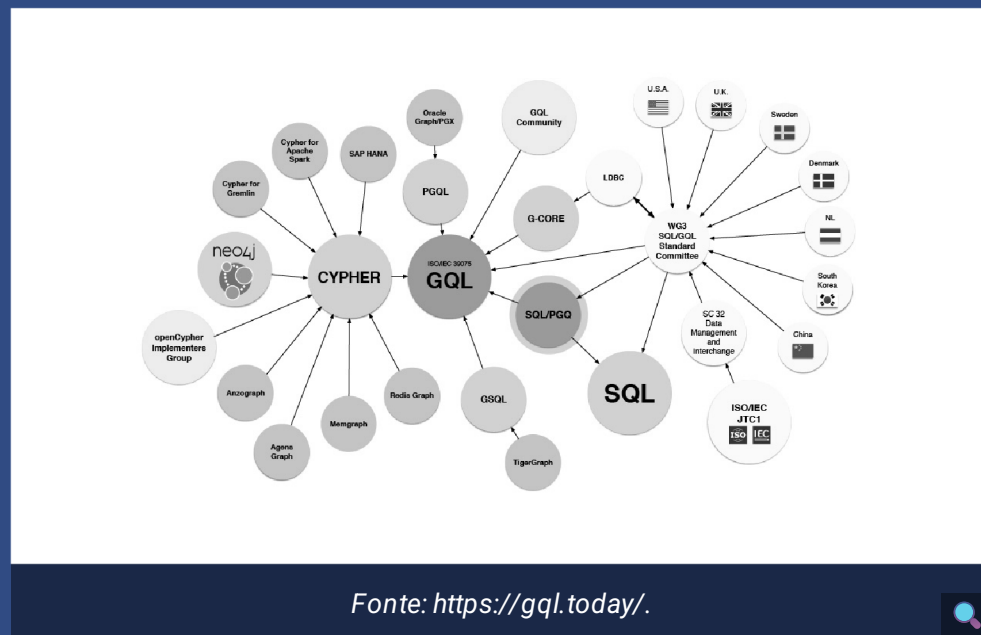
Linguagens de Consulta sobre Grafos (GQL – Graph Query Languages)

A linguagem e protocolo de acesso SPARQL, também padronizada pela W3C, é a linguagem de manipulação para dados em grafo RDF. Trata-se de uma linguagem declarativa (um estilo semelhante ao SQL), para que os usuários possam se concentrar na especificação, e

o mecanismo de consulta é responsável por gerar o melhor plano de execução possível.

A linguagem de consulta é destinada principalmente a recuperar subgrafos usando a correspondência de padrão morfológico, sendo possível especificar variáveis na consulta a serem substituídas por instâncias na resposta. No caso das consultas baseadas em caminhos, é possível especificar um caminho de comprimento arbitrário, com algumas limitações.

No que diz respeito ao modelo LPG, ainda existem algumas linguagens de manipulação possíveis: Cypher, Gremlin, PGQL, GSQL e G-Core. Existe uma iniciativa em andamento para a padronização da GQL, associada ao modelo LPG, ilustrada na figura a seguir:



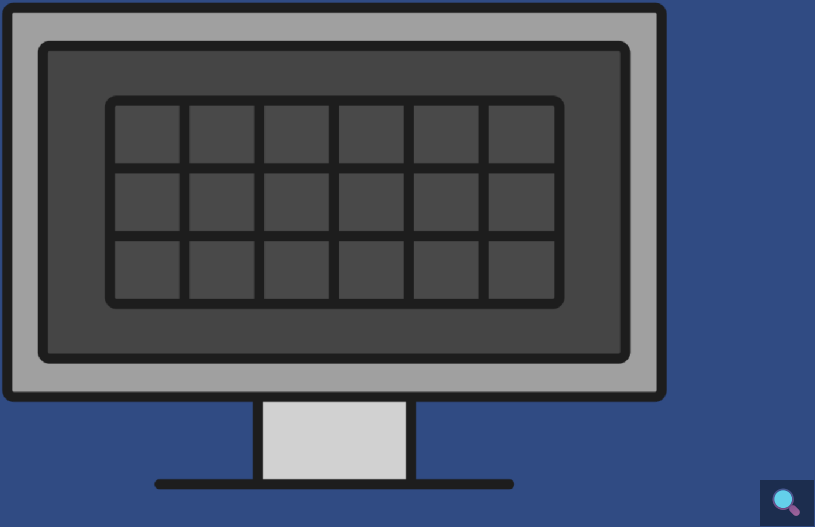
O Neo4j usa a linguagem de consulta Cypher, que foi padronizada sob o nome openCypher; veremos mais detalhes sobre ela posteriormente.

Cabe observar que a linguagem GraphQL, da empresa Facebook, não é uma GQL.

“ A GraphQL é uma linguagem de consulta para APIs e um mecanismo de execução para atender a essas consultas com seus dados existentes. Trata-se de uma especificação normalmente usada para comunicações remotas cliente-servidor. Ao contrário do SQL, a GraphQL é independente da(s) fonte(s) de dados usada(s) para recuperar dados e manter as alterações. O acesso e a manipulação de dados são realizados com funções arbitrárias chamadas resolvers.

Modelos de dados baseados em grafos

As tecnologias de SGBDs relacionais oferecem suporte ao modelo de dados relacional com esquema definido *a priori*, incluindo relações e restrições de integridade, enquanto bancos de dados NoSQL oferecem modelos de dados mais flexíveis, que podem ter múltiplos esquemas ou nem mesmo ter um esquema definido, a fim de reduzir o impacto da evolução do esquema. Assim, o banco de dados sem esquema ou com esquema flexível passa a ser interpretado no nível do aplicativo.



Os SGBDs de grafo têm como foco principal os relacionamentos, enquanto os SGBDs relacionais focam nos dados contidos nas relações (tabelas).

Mas uma pergunta é recorrente:

Será que tabelas poderiam ser usadas para modelar um grafo? E a resposta é: **sim**.

Uma possível solução seria criar um esquema que poderia ser composto por uma tabela para cada tipo de nó, com um ID e seus atributos (propriedades) como colunas, e uma tabela para cada tipo de relacionamento, com o ID dos nós envolvidos (origem e destino) e seus atributos (propriedades) como colunas.

Entretanto, dois aspectos devem ser considerados. Clique:

Interativo

Descrição do interativo

Os bancos de dados em grafo nativos se destacam dos demais pelo conceito de adjacência sem índice. Os relacionamentos são armazenados com os ponteiros para os nós, e cada nó está ciente de todos os relacionamentos de entrada e saída conectados a ele. O mecanismo subjacente simplesmente percorre os ponteiros na memória. Desse modo, os bancos de dados em grafo são preferenciais para armazenamento desse tipo de dados, uma vez que suportam nativamente essas estruturas sem perda de flexibilidade e desempenho.

LPG no sistema Neo4J

O Neo4j é um SGBD de grafos nativo — isto é, ele armazena grafos usando uma adaptação da lista de adjacência e da lista de arestas. Ele foi lançado em 2007 e escrito na linguagem de programação

Java. O Neo4J implementa o modelo de dados LPG e utiliza a linguagem de manipulação de grafos Cypher.

Interativo

Descrição do interativo

Exemplo de manuseio de grafos com Neo4J

Para os exemplos e exercícios, vamos usar um esquema de dados sobre filmes, descrito a seguir:

Todos os nós do tipo Movie têm uma propriedade, *title*, que é usada para identificar exclusivamente um filme. Essa propriedade existe para todos os nós desse tipo. Outras propriedades que um nó Movie pode ter são: (1) *released*, que representa o ano em que o filme foi lançado; (2) *tagline*, que representa uma frase para descrever o filme.

Todos os nós do tipo Person têm uma propriedade, *name*, que é usada para identificar exclusivamente uma pessoa. Alguns nós Person têm uma propriedade *born*.

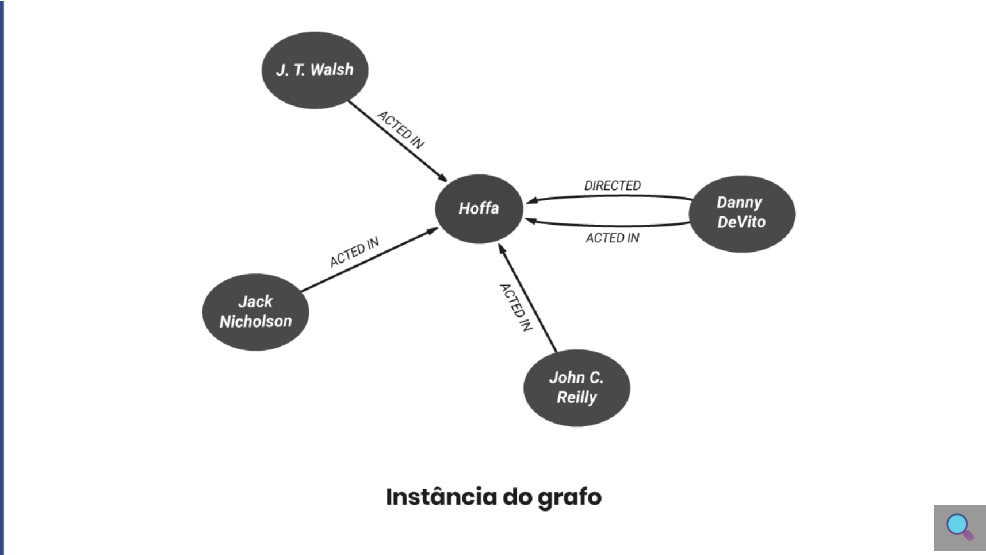
Sobre os relacionamentos nesse modelo, temos:

Interativo

Descrição do interativo

O relacionamento ACTED_IN pode ter a propriedade *role*, que representa os papéis de um ator (pessoa) que atuou em um filme específico. O relacionamento REVIEWED tem as propriedades de *rating* e *summary*.

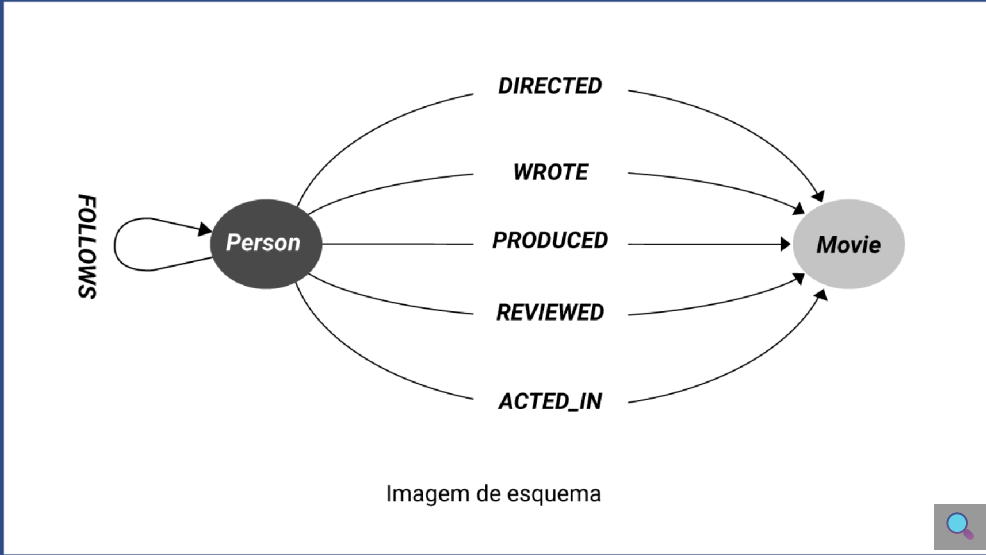
Veja o primeiro exemplo de instância do grafo na imagem a seguir:



Agora, analise um segundo exemplo de instância do grafo na imagem a seguir:



Por fim, confira uma imagem de esquema a seguir:



Cypher no Neo4J

Cypher é uma linguagem de consulta de Grafos fortemente baseada na recuperação de caminhos, mas permite a recuperação de nós e de padrões de subgrafos e a definição de nós, arestas e suas propriedades. A Cypher também é baseada em cláusulas, tendo uma semelhança com SQL.

Quanto às cláusulas da linguagem Cypher para consultas, temos:

MATCH
[WHERE]
RETURN

A cláusula MATCH é obrigatória e permite especificar o padrão do grafo a ser recuperado (de modo semelhante à cláusula FROM da linguagem SQL) — nesse caso, somente os nós. A cláusula WHERE é opcional e permite definir filtros adicionais. A cláusula RETURN especifica o que será recuperado (sendo semelhante ao SELECT na linguagem SQL), sendo possível recuperar nós, relacionamentos e propriedades.

Além disso, uma biblioteca de extensão permite que a linguagem seja estendida, usando-se a linguagem de programação Java ou o próprio Cypher para definir novas funções e procedimentos, que podem ser chamados usando a cláusula CALL. Por exemplo, no Neo4J temos a função db.schema.visualization, que recupera o esquema do grafo armazenado. Trata-se de um recurso equivalente às funções definidas pelo usuário e aos procedimentos armazenados comuns em alguns sistemas de gerenciamento de banco de dados relacional (RDBMSs).

Buscas de nós

Considere o seguinte exemplo de consulta Cypher:

MATCH (p) RETURN p

Esta consulta recupera todos os nós do grafo, independentemente do seu tipo/rótulo. A variável p representa esses nós. Os nós são representados entre parênteses () na cláusula MATCH.

() (p) (:Person) (p:Person)

Veja agora um segundo exemplo:

**MATCH (p:Person)
RETURN p**

Esta consulta recupera todos os nós do grafo do tipo Person. A variável p representa esses nós, e seu tipo é especificado depois dos dois pontos.

Temos a seguir mais um exemplo para recuperação de nós:

**MATCH (p:Person {name: 'Tom Hanks'})
RETURN p**

Propriedades e seus valores são representados entre chaves {} na cláusula MATCH.

{propriedade: valor}

Semelhante à consulta anterior, a próxima consulta recupera todos os nós do grafo do tipo Person e aplica um filtro na propriedade *name* para o valor “Tom Hanks”. Para especificar esse filtro, foi necessário colocar o nome da propriedade e o valor de interesse entre chaves dentro dos parênteses associados ao tipo de nó. Porém alguns filtros são mais complexos; nesse caso, faz-se necessário utilizar a cláusula WHERE, como no exemplo a seguir:

**MATCH (p:Person)
WHERE p.name = 'Tom Hanks' OR p.name = 'Rita Wilson'
RETURN p.name, p.born**

Diferentemente da consulta anterior, o filtro na cláusula WHERE permite combinar, por meio de operadores lógicos AND, OR e NOT, as propriedades de interesse. Além disso, nesse exemplo, a cláusula RETURN não recupera os nós, e sim uma tabela com as duas propriedades de interesse dos nós p selecionados: *name* e *born*.

Recuperando relacionamentos

A consulta a seguir retorna o nome dos filmes em que o ator Tom Hanks atuou:

**MATCH (p:Person {name: 'Tom Hanks'})-[:ACTED_IN]->(m:Movie)
RETURN m.title**

Relacionamentos são representados entre colchetes [] na cláusula MATCH.

() -> () () <- () () - ()
[], [r], [:RELATION], [r:RELATION]

A direção do relacionamento é especificada com os símbolos de maior > e menor < ao lado do nó de destino, sendo opcional. Uma variável pode ser associada ao relacionamento, e o seu tipo pode ser especificado.

Esta outra consulta retorna todos os relacionamentos, independentemente do tipo/rótulo, entre dois nós do tipo Person:

MATCH (p1:Person)-[r]->(p1:Person)
RETURN p1, r, p2

Exemplos de criação de nós

A linguagem Cypher apresenta duas cláusulas possíveis para criação de nós e arestas no grafo: MERGE e CREATE. Em caso de criação de nós, devemos especificar o seu tipo.

MERGE (p:Person {name: 'Michael Cain'})
CREATE (p:Person {name: 'Daniel Kaluuya'})

Observe que, ao usar MERGE para criar um nó, você deve especificar pelo menos uma propriedade que será a chave primária exclusiva do nó. A diferença em relação ao CREATE é que, com essa cláusula, o Neo4J não verifica a chave primária antes de adicionar o nó, então ele deve ser usado somente quando os dados estão íntegros, para garantir maior velocidade durante a importação. Se executarmos os três comandos de criação a seguir, nesta ordem, o Neo4J vai retornar erro para o comando 2.

CREATE (p:Person {name: 'Daniel Kaluuya'})
MERGE (p:Person {name: 'Daniel Kaluuya'})
CREATE (p:Person {name: 'Daniel Kaluuya'})

O uso do MERGE é recomendado para evitar a duplicação de nós. Também podemos encadear várias cláusulas MERGE em um único bloco de código Cypher e recuperar os nós criados:

MERGE (p:Person {name:'Katie Holmes'})
MERGE (m:Movie {title: 'The Dark Knight'})
RETURN p, m

Criação de relacionamentos

Assim como a cláusula MERGE é usada para criar nós no grafo, também pode ser usada para criar relacionamentos entre dois nós. Primeiro, devem ser feitas referências, por meio da cláusula MATCH ou MERGE, aos dois nós para os quais será criado o relacionamento. No comando MERGE de criação dos relacionamentos, deve ser especificado o tipo. A direção do relacionamento pode ser especificada ou não, mas, caso não seja, assume-se que a direção é do nó da esquerda para a direita.

Os exemplos a seguir criam um relacionamento entre o ator p e um filme m:

MATCH (p:Person {name: 'Michael Cain'})
MATCH (m:Movie {title: 'The Dark Knight'})
MERGE (p)-[:ACTED_IN]->(m)

MERGE (p:Person {name: 'Emily Blunt'})-[:ACTED_IN]->(m:Movie {title: 'A Quiet Place'})
RETURN p, m

MERGE (p:Person {name: 'Chadwick Boseman'})
MERGE (m:Movie {title: 'Black Panther'})
MERGE (p)-[:ACTED_IN]-(m)

MATCH (p:Person {name: 'Daniel Kaluuya'})
MERGE (m:Movie {title: 'Get Out'})
MERGE (p) -[:ACTED_IN]-(m)

No primeiro e no segundo, os nós já existem na base, e a direção do relacionamento é especificada. No terceiro, os dois nós são criados e referenciados para a criação do relacionamento, sem especificar a direção, em um mesmo bloco com múltiplos MERGE. O último combina a recuperação de um nó existente para Person e a criação de um novo nó do tipo Movie antes da criação do relacionamento.

Criar, atualizar e remover propriedades

Existem duas formas para adicionar propriedades a nós e arestas: (1) como parte dos comandos MERGE e CREATE e (2) usando a cláusula SET em conjunto com o MATCH. Essa última também é usada para atualizar propriedades existentes.

A seguir, temos um exemplo com o MERGE, que adiciona propriedades para os nós e para a aresta durante a sua criação:

MERGE (p:Person {name: 'Michael Cain'})
MERGE (m:Movie {title: 'Batman Begins'})
MERGE (p) -[:ACTED_IN {roles: ['Alfred Penny']}]->(m)
RETURN p,m

Este outro exemplo permite adicionar ou atualizar o valor de uma propriedade do relacionamento ACTED_IN, usando a cláusula SET (semelhante à linguagem SQL) e informando o nome da propriedade e o seu respectivo valor:

MATCH (p:Person) -[r:ACTED_IN]->(m:Movie)
WHERE p.name = 'Michael Cain' AND m.title = 'The Dark Knight'
SET r.roles = ['Alfred Penny']
RETURN p, r, m

A cláusula REMOVE, em conjunto com o MATCH, permite remover a propriedade de um nó ou aresta, mas não se deve remover a propriedade usada como chave primária de um nó. A seguir, temos um exemplo:

MATCH (p:Person) -[r:ACTED_IN]->(m:Movie)
WHERE p.name = 'Michael Cain' AND m.title = 'The Dark Knight'
REMOVE r.roles
RETURN p, r, m

Também é possível utilizar a cláusula SET para atualizar uma propriedade com o valor NULO, porém é importante especificar qual é a semântica desse valor para a propriedade e como se diferencia dos nós onde a propriedade não existe.

MATCH (p:Person)
WHERE p.name = 'Gene Hackman'
SET p.born = null
RETURN p

Remover nós e relacionamentos

A última operação a ser tratada é a deleção de objetos do grafo. Para remover, devemos usar a cláusula DELETE associada ao MATCH.

O exemplo a seguir deleta um relacionamento r do tipo ACTED_IN entre dois nós p e m selecionados:

MATCH (p:Person {name: 'Jane Doe'}) -[r:ACTED_IN]->(m:Movie {title: 'The Matrix'})

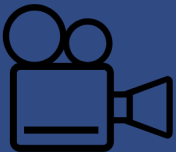
DELETE r
RETURN p, m

O segundo exemplo remove um nó p do tipo Person selecionado pelo filtro da propriedade name e todos os seus relacionamentos, usando a cláusula DETACH antes do DELETE:

MATCH (p:Person {name: 'Jane Doe'})
DETACH DELETE p

O último comando deleta todos os nós e relacionamentos do grafo no banco de dados:

MATCH (n)
DETACH DELETE n



React de mercado

Para encerrarmos o assunto desta aula, acompanhe o *react* de mercado, onde o professor Antony Seabra compartilha um pouco de suas experiências.



Referências

Clique aqui para acessar as referências e créditos desta aula.



ELMASRI, R.; NAVATHE, S. B. **Sistemas de bancos de dados**. 7. ed. São Paulo: Pearson Education do Brasil, 2019.

MONGODB. What is MongoDB. **MongoDB**, c2023. Disponível em: <https://www.mongodb.com/docs/manual>. Acesso em: 23 jun. 2023.

NEO4J. Neo4j Graph Database. **Neo4j**, c2023. Disponível em: <https://neo4j.com/product/neo4j-graph-database/>. Acesso em: 23 jun. 2023.

ÖZSU, T.; VALDURIEZ, P. **Principles of distributed database systems**. 4. ed. New York: Springer, 2019.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistema de bancos de dados**. 7. ed. Rio de Janeiro: LTC, 2020.

Banco de imagens Envato, Freepik, Pexels, Pixabay e Shutterstock.